DESARROLLO DE APLICACIONES CON OBJETOS · TPI · INTERFACES GRÁFICAS

Formularios con Tkinter

Los 6 ejercicios de práctica del material conceptual resueltos, con la captura de cada ventana

Validaciones | Eventos | Cuadros de diálogo | 6 ejercicios probados

Qué hay en esta guía

Los **6 ejercicios de práctica** del material conceptual de interfaces gráficas (Tkinter), resueltos y probados. Cada uno tiene el enunciado tal cual, cómo se piensa, el código completo, una **captura de la ventana** y lo que muestran sus cuadros de diálogo. Las validaciones están en funciones aparte de la ventana, así se pueden probar solas (como pide la guía de Flet: la lógica, comprobable sin abrir la interfaz).

0 Tkinter en una hoja

Para	Se usa
La ventana	`class Formulario(tk.Tk)` + `title()`, `resizable()` y `mainloop()`
Ubicar los controles	`grid(row=…, column=…, sticky="w")` dentro de un `ttk.Frame(padding=16)`
Escribir	`ttk.Entry` (`show="*"` para contraseñas) · `tk.Text` para varias líneas
Elegir	`ttk.Combobox(values=…, state="readonly")` · `ttk.Radiobutton` con una `StringVar` · `ttk.Checkbutton` con una `BooleanVar` cada uno · `ttk.Spinbox`
Leer y cambiar	`entry.get()`, `entry.delete(0, "end")`, `var.get()`, `label.config(text=…)`
Eventos	`command=self.metodo` en botones · `widget.bind("<Return>" | "<FocusOut>" | "<KeyRelease>", …)` · `var.trace_add("write", …)`
Avisar	`messagebox.showinfo / showwarning / showerror(titulo, texto)` · una etiqueta roja debajo del campo

Las capturas

Se sacaron en Linux, por eso los controles se ven con el estilo básico de Tk. En Windows, el mismo código se ve con los controles de Windows.

1 Formulario de Registro de Usuario con Validaciones

Tkinter | Material conceptual · Ejercicio 1 | Solución propuesta

Enunciado (tal cual)

Crea un formulario de registro de usuario que contenga los siguientes campos:

- Nombre de usuario (Entry)
- Correo electrónico (Entry)
- Contraseña (Entry)
- Confirmar contraseña (Entry)
- Botón de "Registrar"

Requisitos:

1. El nombre de usuario debe contener solo letras y números (sin espacios).
2. El correo electrónico debe tener un formato válido (e.g., nombre@dominio.com).
3. La contraseña debe tener al menos 8 caracteres y contener al menos una letra y un número.
4. La contraseña y la confirmación deben coincidir.
5. Cuando se presione el botón "Registrar", debe aparecer un mensaje indicando si la validación fue exitosa o si hubo errores en alguno de los campos.

Cómo se piensa

- Las reglas van en una función aparte, `validar_registro()`, que devuelve la lista de errores: se prueba sin abrir la ventana.
- `isalnum()` ya controla «solo letras y números, sin espacios»; el correo se valida con una expresión regular.
- Las contraseñas usan `show="*"`. Al registrar se muestra `showerror` con todos los errores o `showinfo` si salió bien.

ej1_registro.py

```
"""Ejercicio 1 — Formulario de registro de usuario con validaciones (Tkinter)."""
import re
import tkinter as tk
from tkinter import messagebox, ttk

PATRON_EMAIL = re.compile(r"[\w.+-]+@[\w-]+(\.[\w-]+)+")


def validar_registro(usuario, email, clave, confirmacion):
    """Devuelve la lista de errores (vacía si todo está bien). No depende de la ventana."""
    errores = []
    if not usuario.isalnum():                       # solo letras y números, sin espacios
        errores.append("El nombre de usuario debe tener solo letras y números (sin espacios).")
    if not PATRON_EMAIL.fullmatch(email):
```

```
        errores.append("El correo no tiene un formato válido (ej.: nombre@dominio.com).")
    if len(clave) < 8 or not any(c.isalpha() for c in clave) or not any(c.isdigit() for c in
clave):
        errores.append("La contraseña debe tener al menos 8 caracteres, con letras y números.")
    if clave != confirmacion:
        errores.append("La contraseña y la confirmación no coinciden.")
    return errores


class FormularioRegistro(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Registro de usuario")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Crear cuenta", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, columnspan=2, pady=(0, 10), sticky="w")
        self.usuario = self._campo(marco, "Nombre de usuario", 1)
        self.email = self._campo(marco, "Correo electrónico", 2)
        self.clave = self._campo(marco, "Contraseña", 3, oculto=True)
        self.confirmacion = self._campo(marco, "Confirmar contraseña", 4, oculto=True)
        ttk.Button(marco, text="Registrar", command=self.registrar).grid(row=5, column=0,
columnspan=2, pady=(12, 0), sticky="ew")

    def _campo(self, marco, texto, fila, oculto=False):
        ttk.Label(marco, text=texto).grid(row=fila, column=0, sticky="w", padx=(0, 10), pady=4)
        entrada = ttk.Entry(marco, width=28, show="*" if oculto else "")
        entrada.grid(row=fila, column=1, pady=4)
        return entrada

    def registrar(self):
        errores = validar_registro(self.usuario.get().strip(), self.email.get().strip(),
                                   self.clave.get(), self.confirmacion.get())
        if errores:
            messagebox.showerror("Registro", "Hay errores:\n\n• " + "\n• ".join(errores))
        else:
            messagebox.showinfo("Registro", f"¡Registro exitoso, {self.usuario.get().strip()}!")


if __name__ == "__main__":
    FormularioRegistro().mainloop()
```

Con datos mal cargados, al tocar «Registrar»:



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Error] Registro
Hay errores:

• El nombre de usuario debe tener solo letras y números (sin espacios).
• El correo no tiene un formato válido (ej.: nombre@dominio.com).
• La contraseña debe tener al menos 8 caracteres, con letras y números.
• La contraseña y la confirmación no coinciden.

[Información] Registro
¡Registro exitoso, ana2026!
```

2 Formulario de Inicio de Sesión con Eventos de Teclado

Tkinter | Material conceptual · Ejercicio 2 | Solución propuesta

Enunciado (tal cual)

Crea un formulario de inicio de sesión que tenga los siguientes campos:

- Nombre de usuario (Entry)
- Contraseña (Entry)
- Botón de "Iniciar sesión"

Requisitos:

1. El formulario debe permitir que, al presionar la tecla "Enter", el formulario se envíe automáticamente sin necesidad de presionar el botón.
2. Implementa un evento que, al perder el foco del campo "Nombre de usuario", verifique si este campo está vacío. Si está vacío, debe aparecer un mensaje de advertencia debajo del campo.
3. La contraseña debe ocultarse usando el parámetro show="*" en el campo Entry.

Cómo se piensa

- `self.bind("<Return>", ...)` en la ventana: Enter envía desde cualquier campo.
- `self.usuario.bind("<FocusOut>", self.revisar_usuario)`: al salir del campo vacío aparece el aviso en una etiqueta roja debajo, que se borra cuando se completa.
- El mismo `revisar_usuario()` se reusa al enviar.

ej2_login.py

```python
"""Ejercicio 2 — Formulario de inicio de sesión con eventos de teclado (Tkinter)."""
import tkinter as tk
from tkinter import messagebox, ttk


class FormularioLogin(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Iniciar sesión")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Iniciar sesión", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, sticky="w", pady=(0, 10))
```

```
        ttk.Label(marco, text="Nombre de usuario").grid(row=1, column=0, sticky="w")
        self.usuario = ttk.Entry(marco, width=30)
        self.usuario.grid(row=2, column=0, sticky="ew", pady=(2, 0))
        # Aviso debajo del campo: vacío hasta que el usuario se va del campo sin escribir nada
        self.aviso = ttk.Label(marco, text="", foreground="#c0392b")
        self.aviso.grid(row=3, column=0, sticky="w", pady=(2, 6))
        ttk.Label(marco, text="Contraseña").grid(row=4, column=0, sticky="w")
        self.clave = ttk.Entry(marco, width=30, show="*")    # la contraseña no se ve
        self.clave.grid(row=5, column=0, sticky="ew", pady=(2, 10))
        ttk.Button(marco, text="Iniciar sesión", command=self.iniciar_sesion).grid(row=6,
column=0, sticky="ew")

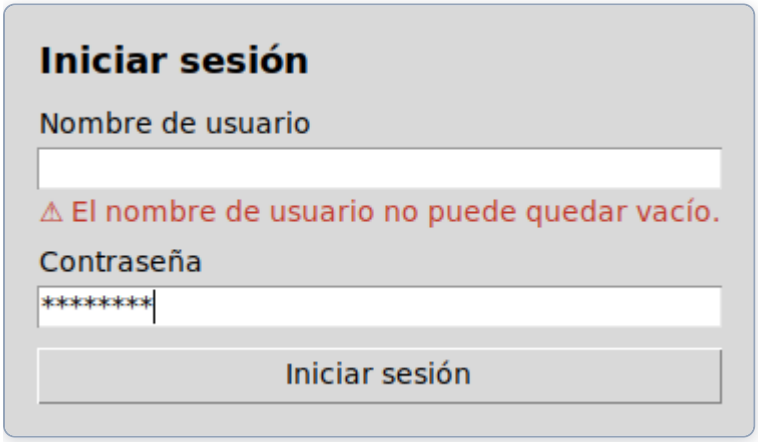
        self.usuario.bind("<FocusOut>", self.revisar_usuario)       # al perder el foco
        self.bind("<Return>", lambda evento: self.iniciar_sesion())  # Enter en cualquier campo

    def revisar_usuario(self, evento=None):
        vacio = not self.usuario.get().strip()
        self.aviso.config(text="⚠ El nombre de usuario no puede quedar vacío." if vacio else "")
        return not vacio

    def iniciar_sesion(self):
        if not self.revisar_usuario() or not self.clave.get():
            messagebox.showwarning("Iniciar sesión", "Completá el usuario y la contraseña.")
            return
        messagebox.showinfo("Iniciar sesión", f"Sesión iniciada como
{self.usuario.get().strip()}.")


if __name__ == "__main__":
    FormularioLogin().mainloop()
```

Después de salir del usuario sin escribir nada:



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Advertencia] Iniciar sesión
Completá el usuario y la contraseña.

[Información] Iniciar sesión
Sesión iniciada como ana2026.
```

3 Formulario de Datos Personales con Validación de Edad

Tkinter | Material conceptual · Ejercicio 3 | Solución propuesta

Enunciado (tal cual)

Crea un formulario que recoja los siguientes datos personales:

- Nombre completo (Entry)
- Edad (Entry)
- País (Combobox)
- Género (RadioButtons)
- Botón de "Enviar"

Requisitos:

1. Validar que la edad ingresada sea un número mayor de 18.
2. Implementa un evento que muestre un mensaje de advertencia cuando la edad no sea un número válido.
3. Al hacer clic en "Enviar", muestra un cuadro de diálogo con todos los datos ingresados y valida que los campos no estén vacíos.

Cómo se piensa

- La edad se revisa en cada tecla (`<KeyRelease>`) y al salir del campo; si no es un número, aparece el aviso debajo.
- Tomé «mayor de 18» como **18 o más** (mayor de edad). Si lo quieren estricto, se cambia `>= 18` por `> 18`.
- El país es un `Combobox` de solo lectura y el género, tres `Radiobutton` con la misma `StringVar`.

ej3_datos_personales.py

```
"""Ejercicio 3 — Formulario de datos personales con validación de edad (Tkinter)."""
import tkinter as tk
from tkinter import messagebox, ttk

PAISES = ["Argentina", "Bolivia", "Brasil", "Chile", "Paraguay", "Uruguay", "Otro"]
GENEROS = ["Femenino", "Masculino", "Otro"]


def edad_valida(texto):
    """Edad: un número entero de 18 o más."""
    return texto.isdigit() and int(texto) >= 18


def validar_datos(nombre, edad, pais, genero):
    errores = []
    if not nombre:
        errores.append("Falta el nombre completo.")
    if not edad:
        errores.append("Falta la edad.")
    elif not edad_valida(edad):
        errores.append("La edad tiene que ser un número de 18 o más.")
    if not pais:
        errores.append("Falta elegir el país.")
    if not genero:
```

```
        errores.append("Falta elegir el género.")
    return errores


class FormularioDatos(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Datos personales")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Datos personales", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, columnspan=2, sticky="w", pady=(0, 10))
        ttk.Label(marco, text="Nombre completo").grid(row=1, column=0, sticky="w", pady=4)
        self.nombre = ttk.Entry(marco, width=28)
        self.nombre.grid(row=1, column=1, pady=4)
        ttk.Label(marco, text="Edad").grid(row=2, column=0, sticky="w", pady=4)
        self.edad = ttk.Entry(marco, width=28)
        self.edad.grid(row=2, column=1, pady=4)
        self.aviso_edad = ttk.Label(marco, text="", foreground="#c0392b")
        self.aviso_edad.grid(row=3, column=1, sticky="w")
        ttk.Label(marco, text="País").grid(row=4, column=0, sticky="w", pady=4)
        self.pais = ttk.Combobox(marco, values=PAISES, state="readonly", width=26)
        self.pais.grid(row=4, column=1, pady=4)
        ttk.Label(marco, text="Género").grid(row=5, column=0, sticky="w", pady=4)
        self.genero = tk.StringVar(value="")
        radios = ttk.Frame(marco)
        radios.grid(row=5, column=1, sticky="w")
        for g in GENEROS:
            ttk.Radiobutton(radios, text=g, value=g, variable=self.genero).pack(side="left",
padx=(0, 8))
        ttk.Button(marco, text="Enviar", command=self.enviar).grid(row=6, column=0, columnspan=2,
sticky="ew", pady=(12, 0))

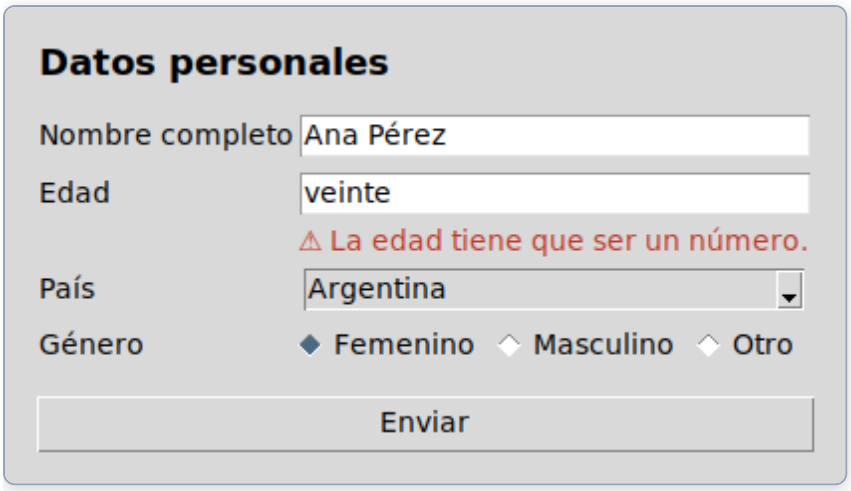
        # Evento: cada vez que se escribe en la edad (o se sale del campo) se revisa
        self.edad.bind("<KeyRelease>", self.revisar_edad)
        self.edad.bind("<FocusOut>", self.revisar_edad)

    def revisar_edad(self, evento=None):
        texto = self.edad.get().strip()
        if texto and not texto.isdigit():
            self.aviso_edad.config(text="⚠ La edad tiene que ser un número.")
        elif texto and int(texto) < 18:
            self.aviso_edad.config(text="⚠ Tiene que ser 18 o más.")
        else:
            self.aviso_edad.config(text="")

    def enviar(self):
        datos = (self.nombre.get().strip(), self.edad.get().strip(), self.pais.get(),
self.genero.get())
        errores = validar_datos(*datos)
        if errores:
            messagebox.showwarning("Datos personales", "\n".join(errores))
            return
        nombre, edad, pais, genero = datos
        messagebox.showinfo("Datos personales", f"Nombre: {nombre}\nEdad: {edad}\nPaís:
{pais}\nGénero: {genero}")


if __name__ == "__main__":
    FormularioDatos().mainloop()
```

Escribiendo «veinte» en la edad:



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Advertencia] Datos personales
La edad tiene que ser un número de 18 o más.

[Información] Datos personales
Nombre: Ana Pérez
Edad: 20
País: Argentina
Género: Femenino
```

4 Formulario de Reserva de Hotel con Validación de Fechas

Tkinter · Material conceptual · Ejercicio 4 · Solución propuesta

Enunciado (tal cual)

Crea un formulario para una reserva de hotel que contenga:

- Nombre completo (Entry)
- Fecha de check-in (Entry con formato dd/mm/yyyy)
- Fecha de check-out (Entry con formato dd/mm/yyyy)
- Número de personas (Spinbox)
- Botón de "Reservar"

Requisitos:

1. Validar que las fechas de check-in y check-out tengan el formato correcto y que la fecha de check-out sea posterior a la de check-in.
2. Validar que el número de personas sea mayor a 0.
3. Al hacer clic en "Reservar", mostrar una ventana emergente que confirme si los datos son correctos o si hubo algún error en la validación.

Cómo se piensa

- `datetime.strptime(texto, "%d/%m/%Y")` controla el formato y que la fecha exista (el 31/02 da error); si falla, la fecha es inválida.
- Con las dos fechas válidas se comparan: el check-out tiene que ser mayor. La confirmación calcula las noches.

- El `Spinbox` va de 0 a 10 y se valida que sea mayor a 0.

ej4_reserva_hotel.py

```
"""Ejercicio 4 — Formulario de reserva de hotel con validación de fechas (Tkinter)."""
import tkinter as tk
from datetime import datetime
from tkinter import messagebox, ttk

FORMATO = "%d/%m/%Y"


def leer_fecha(texto):
    """Devuelve la fecha si tiene el formato dd/mm/aaaa y existe; si no, None."""
    try:
        return datetime.strptime(texto, FORMATO).date()
    except ValueError:
        return None


def validar_reserva(nombre, check_in, check_out, personas):
    errores = []
    if not nombre:
        errores.append("Falta el nombre completo.")
    entrada, salida = leer_fecha(check_in), leer_fecha(check_out)
    if entrada is None:
        errores.append("La fecha de check-in debe tener el formato dd/mm/aaaa.")
    if salida is None:
        errores.append("La fecha de check-out debe tener el formato dd/mm/aaaa.")
    if entrada and salida and salida <= entrada:
        errores.append("El check-out tiene que ser posterior al check-in.")
    if not personas.isdigit() or int(personas) <= 0:
        errores.append("El número de personas tiene que ser mayor a 0.")
    return errores

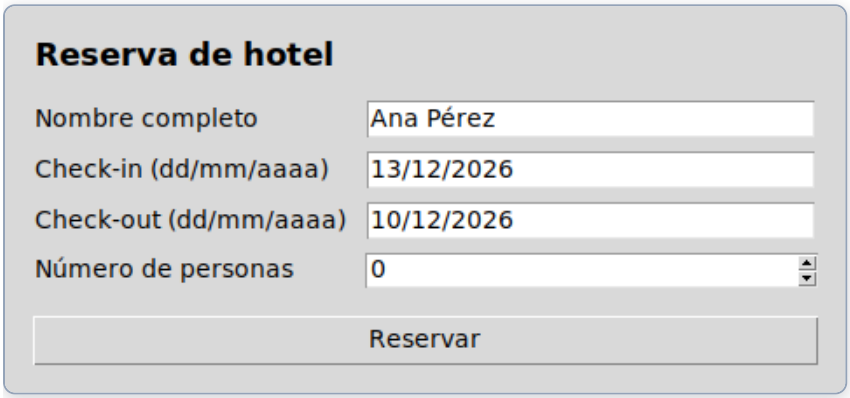

class FormularioReserva(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Reserva de hotel")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Reserva de hotel", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, columnspan=2, sticky="w", pady=(0, 10))
        self.nombre = self._campo(marco, "Nombre completo", 1)
        self.check_in = self._campo(marco, "Check-in (dd/mm/aaaa)", 2)
        self.check_out = self._campo(marco, "Check-out (dd/mm/aaaa)", 3)
        ttk.Label(marco, text="Número de personas").grid(row=4, column=0, sticky="w", pady=4)
        self.personas = ttk.Spinbox(marco, from_=0, to=10, width=26)
        self.personas.set(1)
        self.personas.grid(row=4, column=1, pady=4)
        ttk.Button(marco, text="Reservar", command=self.reservar).grid(row=5, column=0,
columnspan=2, sticky="ew", pady=(12, 0))

    def _campo(self, marco, texto, fila):
        ttk.Label(marco, text=texto).grid(row=fila, column=0, sticky="w", padx=(0, 10), pady=4)
        entrada = ttk.Entry(marco, width=28)
        entrada.grid(row=fila, column=1, pady=4)
        return entrada
```

```
    def reservar(self):
        datos = (self.nombre.get().strip(), self.check_in.get().strip(),
self.check_out.get().strip(), self.personas.get().strip())
        errores = validar_reserva(*datos)
        if errores:
            messagebox.showerror("Reserva", "No se pudo reservar:\n\n• " + "\n• ".join(errores))
            return
        noches = (leer_fecha(datos[2]) - leer_fecha(datos[1])).days
        messagebox.showinfo("Reserva", f"Reserva confirmada para {datos[0]}:\n{datos[1]} al
{datos[2]} "
                                       f"({noches} {'noche' if noches == 1 else 'noches'}),
{datos[3]} persona(s).")


if __name__ == "__main__":
    FormularioReserva().mainloop()
```

Con el check-out antes del check-in y 0 personas:



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Error] Reserva
No se pudo reservar:

• El check-out tiene que ser posterior al check-in.
• El número de personas tiene que ser mayor a 0.

[Información] Reserva
Reserva confirmada para Ana Pérez:
10/12/2026 al 13/12/2026 (3 noches), 2 persona(s).
```

5 Formulario de Contacto con Eventos Dinámicos

Tkinter | Material conceptual · Ejercicio 5 | Solución propuesta

Enunciado (tal cual)

Crea un formulario de contacto que contenga:

- Nombre (Entry)
- Correo electrónico (Entry)
- Mensaje (Text widget)
- Botón de "Enviar"

Requisitos:

1. El campo "Correo electrónico" debe validarse en tiempo real: cuando el usuario escriba, si el correo no tiene el formato correcto, debe aparecer una advertencia en color rojo debajo del campo.
2. El botón "Enviar" solo debe habilitarse si todos los campos han sido llenados correctamente.
3. Al hacer clic en "Enviar", mostrar un cuadro de diálogo que confirme que el mensaje ha sido enviado.

Cómo se piensa

- Las entradas usan `StringVar` con `trace_add("write", ...)`: cada letra llama a `actualizar()`. El `Text` no tiene variable, por eso usa `<KeyRelease>`.
- `actualizar()` muestra u oculta la advertencia roja y habilita el botón solo si todo está bien (`state="normal"` o `"disabled"`).

ej5_contacto.py

```
"""Ejercicio 5 — Formulario de contacto con eventos dinámicos (Tkinter)."""
import re
import tkinter as tk
from tkinter import messagebox, ttk

PATRON_EMAIL = re.compile(r"[\w.+-]+@[\w-]+(\.[\w-]+)+")


def email_valido(texto):
    return bool(PATRON_EMAIL.fullmatch(texto))


def formulario_completo(nombre, email, mensaje):
    return bool(nombre.strip()) and email_valido(email.strip()) and bool(mensaje.strip())


class FormularioContacto(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Contacto")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Contacto", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, sticky="w", pady=(0, 10))
        self.nombre = tk.StringVar()
```

```
        self.email = tk.StringVar()
        ttk.Label(marco, text="Nombre").grid(row=1, column=0, sticky="w")
        ttk.Entry(marco, textvariable=self.nombre, width=40).grid(row=2, column=0, sticky="ew",
pady=(2, 8))
        ttk.Label(marco, text="Correo electrónico").grid(row=3, column=0, sticky="w")
        ttk.Entry(marco, textvariable=self.email, width=40).grid(row=4, column=0, sticky="ew",
pady=(2, 0))
        self.aviso = tk.Label(marco, text="", fg="red")           # advertencia en rojo debajo
del campo
        self.aviso.grid(row=5, column=0, sticky="w", pady=(0, 6))
        ttk.Label(marco, text="Mensaje").grid(row=6, column=0, sticky="w")
        self.mensaje = tk.Text(marco, width=40, height=5, relief="solid", borderwidth=1)
        self.mensaje.grid(row=7, column=0, sticky="ew", pady=(2, 10))
        self.boton = ttk.Button(marco, text="Enviar", command=self.enviar, state="disabled")
        self.boton.grid(row=8, column=0, sticky="ew")

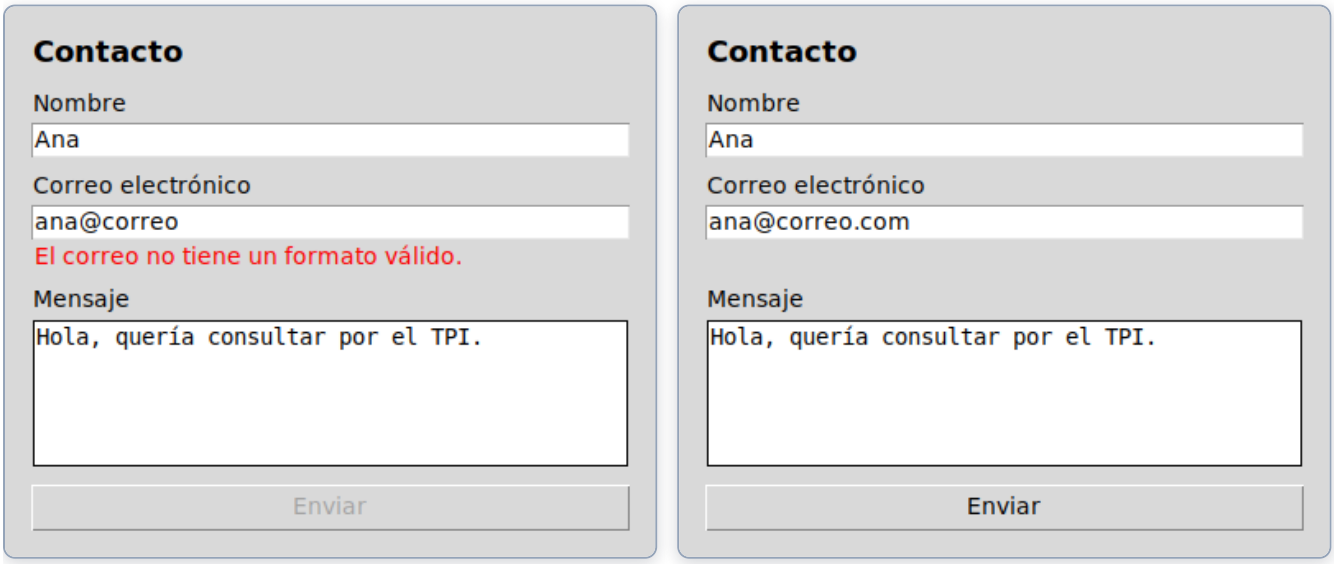
        # Validación en tiempo real: trace sobre las variables y KeyRelease sobre el Text
        self.nombre.trace_add("write", self.actualizar)
        self.email.trace_add("write", self.actualizar)
        self.mensaje.bind("<KeyRelease>", self.actualizar)

    def actualizar(self, *args):
        texto = self.email.get().strip()
        self.aviso.config(text="El correo no tiene un formato válido." if texto and not
email_valido(texto) else "")
        listo = formulario_completo(self.nombre.get(), self.email.get(), self.mensaje.get("1.0",
"end"))
        self.boton.config(state="normal" if listo else "disabled")    # solo se habilita si todo
está bien

    def enviar(self):
        messagebox.showinfo("Contacto", f"¡Gracias, {self.nombre.get().strip()}! Tu mensaje fue
enviado.")


if __name__ == "__main__":
    FormularioContacto().mainloop()
```

Con el correo incompleto el botón queda deshabilitado; al corregirlo se habilita:



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Información] Contacto
¡Gracias, Ana! Tu mensaje fue enviado.
```

6 Formulario de Encuesta con Selección Múltiple y Validación

Tkinter | Material conceptual · Ejercicio 6 | Solución propuesta

Enunciado (tal cual)

Crea un formulario de encuesta que permita a los usuarios seleccionar varias opciones:

- Nombre (Entry)
- Género (RadioButtons)
- Opciones de preferencia (Checkbuttons) [Ej: Opción 1, Opción 2, Opción 3]
- Botón de "Enviar"

Requisitos:

1. El campo "Nombre" no debe quedar vacío, y debe validarse que el usuario seleccione al menos una opción en las "Opciones de preferencia".
2. Cuando el formulario se envíe, mostrar los datos ingresados y las opciones seleccionadas en un cuadro de diálogo.
3. Si algún campo no es completado, resaltar los campos faltantes con un borde rojo.

Cómo se piensa

- El borde rojo sale de `highlightbackground`: por eso el nombre es un `tk.Entry` (el de ttk no lo tiene) y cada grupo de botones va dentro de un `Frame` con borde.
- Cada `Checkbutton` tiene su `BooleanVar`; las elegidas son las que están en `True`.
- Al enviar, se pinta de rojo lo que falta y se vuelve al gris lo que ya está bien.

ej6_encuesta.py

```python
"""Ejercicio 6 — Formulario de encuesta con selección múltiple y validación (Tkinter)."""
import tkinter as tk
from tkinter import messagebox, ttk

GENEROS = ["Femenino", "Masculino", "Otro"]
OPCIONES = ["Opción 1", "Opción 2", "Opción 3"]
ROJO, NORMAL = "#e74c3c", "#c8c8c8"


def campos_faltantes(nombre, genero, elegidas):
    """Devuelve los nombres de los campos que faltan completar."""
    faltan = []
    if not nombre.strip():
        faltan.append("nombre")
    if not genero:
        faltan.append("genero")
    if not elegidas:
        faltan.append("opciones")
    return faltan
```

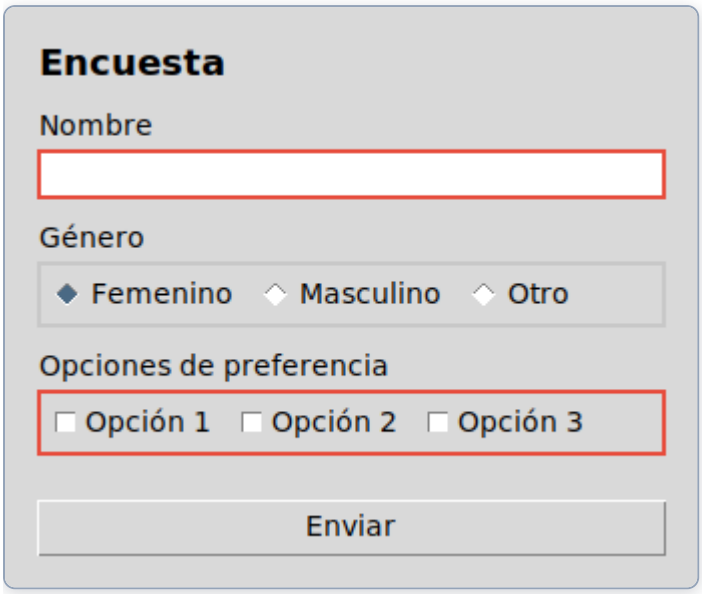
```
class FormularioEncuesta(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Encuesta")
        self.resizable(False, False)
        marco = ttk.Frame(self, padding=16)
        marco.pack()
        ttk.Label(marco, text="Encuesta", font=("TkDefaultFont", 13, "bold")).grid(row=0,
column=0, sticky="w", pady=(0, 10))
        ttk.Label(marco, text="Nombre").grid(row=1, column=0, sticky="w")
        # tk.Entry (no ttk) porque permite pintar el borde con highlightbackground
        self.nombre = tk.Entry(marco, width=34, highlightthickness=2, highlightbackground=NORMAL,
relief="flat")
        self.nombre.grid(row=2, column=0, sticky="w", pady=(2, 10))
        # Los grupos van dentro de un Frame con borde, para poder marcarlos en rojo
        self.genero = tk.StringVar(value="")
        self.marco_genero = self._grupo(marco, "Género", 3)
        for g in GENEROS:
            ttk.Radiobutton(self.marco_genero, text=g, value=g,
variable=self.genero).pack(side="left", padx=6, pady=4)
        self.marco_opciones = self._grupo(marco, "Opciones de preferencia", 5)
        self.opciones = {o: tk.BooleanVar() for o in OPCIONES}
        for o, var in self.opciones.items():
            ttk.Checkbutton(self.marco_opciones, text=o, variable=var).pack(side="left", padx=6,
pady=4)
        ttk.Button(marco, text="Enviar", command=self.enviar).grid(row=7, column=0, sticky="ew",
pady=(12, 0))

    def _grupo(self, marco, titulo, fila):
        ttk.Label(marco, text=titulo).grid(row=fila, column=0, sticky="w")
        grupo = tk.Frame(marco, highlightthickness=2, highlightbackground=NORMAL)
        grupo.grid(row=fila + 1, column=0, sticky="ew", pady=(2, 10))
        return grupo

    def enviar(self):
        elegidas = [o for o, var in self.opciones.items() if var.get()]
        faltan = campos_faltantes(self.nombre.get(), self.genero.get(), elegidas)
        # Borde rojo en lo que falta y normal en lo demás
        for campo, widget in (("nombre", self.nombre), ("genero", self.marco_genero),
("opciones", self.marco_opciones)):
            widget.config(highlightbackground=ROJO if campo in faltan else NORMAL)
        if faltan:
            messagebox.showwarning("Encuesta", "Completá los campos marcados en rojo (elegí al
menos una opción).")
            return
        messagebox.showinfo("Encuesta", f"Nombre: {self.nombre.get().strip()}\nGénero:
{self.genero.get()}\n"
                                        f"Opciones: {', '.join(elegidas)}")


if __name__ == "__main__":
    FormularioEncuesta().mainloop()
```

Enviando sin nombre y sin opciones (el género sí está elegido):



Lo que muestran los cuadros de diálogo (con datos mal y con datos bien)

```
[Advertencia] Encuesta
Completá los campos marcados en rojo (elegí al menos una opción).

[Información] Encuesta
Nombre: Ana Pérez
Género: Femenino
Opciones: Opción 1, Opción 3
```